

Towards Fully Automated Medical Imaging Code Generation via Validation-Based Context Engineering

Zixiao Zhao

University of Auckland
Auckland, New Zealand
zixiao.zhao@auckland.ac.nz

Jing Sun

University of Auckland
Auckland, New Zealand
jing.sun@auckland.ac.nz

Zhe Hou

Griffith University
Brisbane, Australia
z.hou@griffith.edu.au

Cheng-Hao Cai

Suzhou Industrial Park Monash
Research Institute of Science and
Technology
Suzhou, China
chenghao.cai.cs@gmail.com

Qian Liu

University of Auckland
Auckland, New Zealand
liu.qian@auckland.ac.nz

Mengze Li

University of Auckland
Auckland, New Zealand
mli840@aucklanduni.ac.nz

Zijian Zhang*

Beijing Institute of Technology
Beijing, China
zhangzijian@bit.edu.cn

Jin Song Dong

National University of Singapore
Singapore, Singapore
dcsdjs@nus.edu.sg

Abstract

Large language models (LLMs) have demonstrated considerable promise in program generation for small-scale and conventional application development; however, they remain limited when applied to complex, domain-specific tasks such as medical image processing. General-purpose models lack explicit domain knowledge and robust validation mechanisms to ensure correctness, often requiring substantial human intervention to produce reliable processing pipelines. To address these limitations, we propose AutoMedImg, a multi-agent framework for fully automated medical image processing code generation. AutoMedImg orchestrates specialised agents across two phases: a Planning Phase that performs dataset analysis and architecture design with semantic and formal verification, and a Coding Phase that generates modules in parallel with static checking, execution testing, and assembly validation. This multi-stage validation mitigates error propagation throughout generation, while comprehensive auto-context engineering combining domain-specific knowledge bases, shared memory, and validation feedback automates context construction without manual prompting. A cross-project adaptive pipeline synthesis mechanism further accumulates validated pipelines and retrieves proven components for new tasks based on project similarity, enhancing generation efficiency through cross-project learning. Extensive evaluation across six diverse and well-established medical imaging datasets with five backbone LLMs demonstrates that AutoMedImg achieves zero human intervention, with Dice scores of up to 0.90 for segmentation tasks and 99% accuracy for classification.

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3834399>

CCS Concepts

• **Software and its engineering** → **Automatic programming**; • **Computing methodologies** → **Multi-agent systems**; • **Applied computing** → *Health care information systems*.

Keywords

Automatic Programming, Large Language Models, Medical Imaging

ACM Reference Format:

Zixiao Zhao, Jing Sun, Zhe Hou, Cheng-Hao Cai, Qian Liu, Mengze Li, Zijian Zhang, and Jin Song Dong. 2026. Towards Fully Automated Medical Imaging Code Generation via Validation-Based Context Engineering. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3834399>

1 Introduction

The emergence of large language models (LLMs) has revolutionised the field of software engineering [14, 19, 21]. Trained on massive code datasets, LLMs show impressive capabilities in natural language requirements comprehension [2], relevant codebases search [26], reasoning on software structures [34], and demonstrating substantial code generation capabilities [7]. On function-level code generation benchmarks such as HumanEval [5] and MBPP [3], state-of-the-art LLMs have achieved accuracy rates of 97.6% and 92.7%, respectively [44, 48]. On SWE-bench [20], a benchmark on solving real-world Github repository issues, LLMs have achieved a 76.8% resolution rate [45]. These results highlight the significant potential of LLMs in software engineering.

However, these benchmarks only demonstrate the capability of LLM systems in small-scale code generation or code repair within existing repositories. When generating complex code from scratch, current LLMs face significant limitations: (1) For complex problems, users must provide highly detailed prompts encompassing all task-relevant background information to achieve satisfactory

results [27]; (2) Due to the hallucination problem inherent in LLMs and the lack of effective validation mechanisms, human intervention remains necessary during code generation [43]; (3) Without built-in validation mechanisms, LLMs cannot maintain consistent context and memory across code generation projects, even when successfully generating complete code [46]. Medical image processing code generation significantly exemplifies these challenges. Medical images exhibit complex properties with multimodal data (CT, MRI), 2D/3D volumetric structures, various intensity ranges, and specialised formats (NIfTI, DICOM) [28, 36], demanding domain expertise that significantly exceeds what can be conveyed through manual prompting. Domain reliability requirements mean that hallucinated preprocessing steps or incorrect architectural choices are unacceptable, necessitating rigorous validation throughout generation. Complete pipelines span multiple interdependent stages from data loading through inference, while recurring patterns across projects demand cross-project knowledge retention that standard LLMs cannot exploit. Automating this process would enable practitioners lacking deep learning expertise to deploy validated medical imaging pipelines, broadening access to AI-assisted diagnostics. Moreover, established benchmarking datasets with ground truth annotations and standardised metrics enable objective assessment of generated pipelines. Together, these properties make medical image processing both a practically impactful and methodologically rigorous domain for evaluating automated code generation in complex, real-world settings.

Although several approaches attempt to tackle these challenges, they remain insufficient. Existing code generation frameworks or tools [11, 15, 47, 52] lack domain-specific knowledge integration and systematic validation mechanisms, making human intervention indispensable for specialised domains. Recent AutoML and multi-agent approaches [9, 49] automate broader ML pipelines but remain confined or pre-defined datasets, requiring manual effort to extend to specialised domains or datasets. Medical imaging automation methods [18, 31, 50] incorporate domain expertise but address only partial pipeline components without achieving fully automated generation. No existing approach combines comprehensive domain knowledge, systematic validation, and consistent context management to enable fully automated generation of complete, validated pipelines from scratch.

To address these limitations, we propose AutoMedImg, a multi-agent framework for automated medical image processing code generation via validation-based context engineering. AutoMedImg integrates domain knowledge bases eliminating exhaustive manual prompting, systematic multi-stage validation preventing error propagation, and adaptive pipeline synthesis maintaining context across projects. Built on a multi-agent architecture, AutoMedImg orchestrates the pipeline from dataset analysis through architecture design, parallel module implementation, and final assembly. Evaluation on six medical imaging datasets demonstrates that AutoMedImg achieves fully automated code generation while maintaining competitive model performance.

The main contributions of this work are as follows:

- (1) We propose a **validation-based context engineering approach** integrating medical imaging domain knowledge, shared memory, and multi-stage validation to automatically

construct and maintain generation context, enabling error detection and correction without manual prompting.

- (2) We develop an **adaptive pipeline synthesis mechanism** that transfers validated medical imaging pipeline components across projects, enabling efficient knowledge reuse while maintaining pipeline quality.
- (3) We present AutoMedImg, a multi-agent framework for **fully automated medical image processing code generation**, evaluated on six diverse datasets, achieving zero human intervention with competitive model performance.

The remainder of this paper is organised as follows: Section 2 reviews related work on automated code generation and medical imaging automation. Section 3 presents the AutoMedImg framework, detailing the multi-agent architecture, systematic validation mechanisms, and validation-guided context engineering. Section 4 evaluates existing state-of-the-art code generation approaches on medical image processing to identify their limitations, and demonstrates how AutoMedImg’s validation-based design addresses these limitations across six medical imaging datasets with ablation studies. Section 5 presents a qualitative comparison with medical imaging automation methods and discusses threats to validity. Section 6 concludes with a summary and future research directions.

2 Related Work

Automated Code Generation with LLMs. LLM-based code generation has evolved from early models like InCoder [10] and CodeGen [38] to proprietary models such as GPT [39], Claude [1], and Gemini [8] achieving state-of-the-art performance on benchmarks like HumanEval [5] and MBPP [3]. Self-debugging frameworks [6] and execution-feedback methods [37] enable refinement based on runtime errors but rely on human-provided test cases. Multi-agent frameworks such as ChatDev [40] and AgentCoder [17] introduced dynamic test generation through agent collaboration, yet remain limited to function-level or module-level generation. Contemporary tools including GitHub Copilot [11], Cursor [47], and Anti-gravity [12] integrate LLMs into IDEs with execution capabilities but still require substantial human intervention across complex multi-stage tasks in specialised domains.

Automation in Medical Image Processing. In the field of medical imaging, numerous automated algorithm design and optimization methods have been developed. NAS [30, 42] automatically discovers optimal network structures but requires predefined search spaces and manual pipeline implementation [50]. nnU-Net [18] automates segmentation pipeline configuration but requires manual environment setup and is restricted to segmentation tasks. Foundation models like SAM [22, 41] and MedSAM [31] eliminate task-specific training but require interactive prompts during inference and address only segmentation. AutoML-Agent [49] automates full-pipeline ML across diverse tasks but requires manually provided skeleton code for new task types and relies solely on prompt-based rather than execution-grounded verification. DS-Agent [13] leverages case-based reasoning for data science automation but lacks medical imaging knowledge for generating appropriate pipelines, and its sole reliance on execution feedback can only guarantee runnable code without ensuring alignment with domain conventions. VoxelPrompt [16] integrates a jointly-trained vision-language

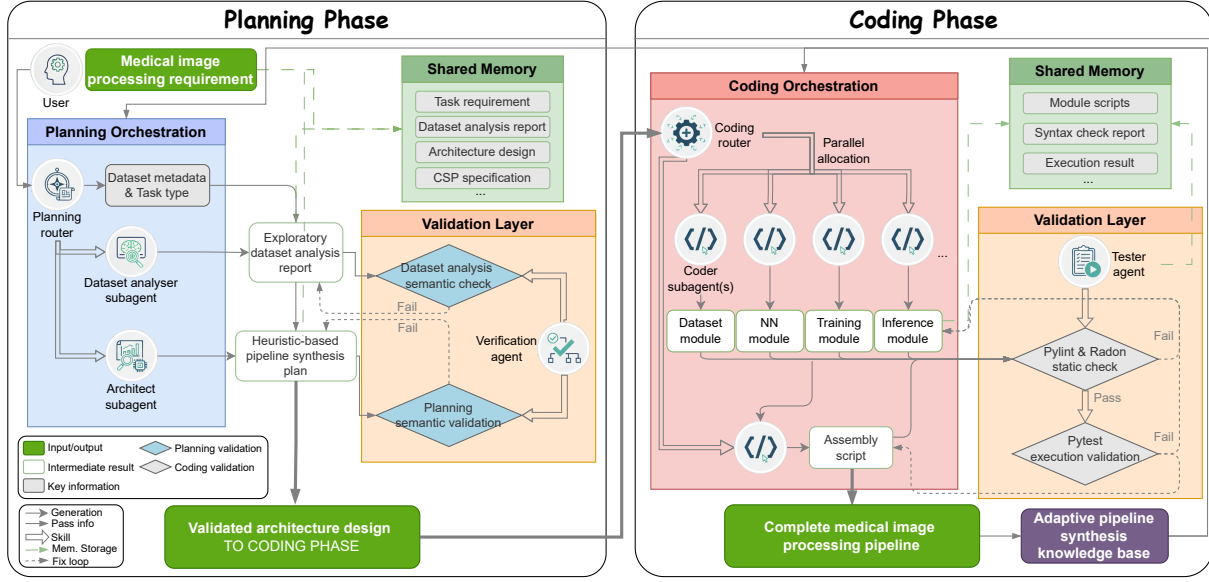


Figure 1: AutoMedImg's two-phase workflow: Planning and Coding

agent for end-to-end radiological analysis but focuses exclusively on inference-time task execution without addressing automated pipeline development or code generation. M³Builder [9] automates medical imaging ML workflows through multi-agent coordination but restricts generation to pre-defined datasets with accompanying code templates, requiring manual preparation for new datasets. These approaches address only partial pipeline components, requiring substantial expertise for deployment, and none achieves fully automated generation of complete pipelines from scratch.

3 Method: AutoMedImg Framework

In this section, we present AutoMedImg workflow and framework through 3 components: Section 3.1 details agent specifications and context integration mechanisms; Section 3.2 explains validation-based context engineering in the Planning and Coding phases; Section 3.3 presents cross-project adaptive pipeline synthesis.

Medical imaging pipelines require understanding the dataset characteristics before implementation. AutoMedImg addresses this through a two-phase architecture, Planning and Coding, as illustrated in Figure 1. **Planning Phase.** Given a medical image processing requirement, the Planning Router first invokes the Dataset Analyser subagent to perform exploratory dataset analysis. The Verification Agent validates this analysis through semantic checking, ensuring all required dataset features are extracted; failures trigger re-analysis with diagnostic feedback. Next, the Architect subagent synthesises a pipeline design based on validated dataset characteristics. This design undergoes planning semantic validation at two levels: contextual validation checks architecture-dataset compatibility, modularity requirements, and medical imaging conventions; formal validation converts the architecture to CSP# specification for PAT-based model checking, ensuring process correctness. Throughout this process, intermediate results (dataset analysis report, architecture design, CSP specification) are stored in Shared Memory for agent coordination. Only architectures passing semantic validation

propagate to the Coding Phase as validated pipeline designs. **Coding Phase.** The Coding Router decomposes the validated pipeline design and allocates multiple Coder subagents for parallel module generation (Dataset, NN, Training, Inference modules). The Tester Agent validates each module through static checking (Pylint, Radon) and execution testing (Pytest); failures trigger code refinement with diagnostic feedback. Validated modules are integrated into an assembly script, which is then subjected to final end-to-end validation. Successfully validated pipelines that achieve satisfactory performance are entered into the Adaptive Pipeline Synthesis Knowledge Base, along with their project characteristics and performance metrics, enabling future projects to retrieve and adapt proven components from similar tasks. This two-phase architecture with systematic multi-stage validation enables fully automated generation of production-ready medical imaging pipelines.

3.1 AutoMedImg Agent Specification

3.1.1 Agent Coordination Structure. Complex medical image processing tasks require comprehensive expertise over multiple domains, including medical image data analysis, library dependencies and APIs, data loading strategies, and neural network architecture design. A single-agent approach faces fundamental limitations: (1) simultaneously invoking knowledge across diverse domains creates confusion and inconsistent reasoning; (2) maintaining accurate context becomes increasingly difficult as context accumulates through planning, validation, and coding phases; (3) single-agent systems cannot exploit parallel execution for efficiency improvement.

AutoMedImg addresses these limitations through a multi-agent architecture organised as a router-subagent hierarchy with an independent Validation Layer. To distribute domain expertise, AutoMedImg employs specialised agents: the Planning Router invokes Dataset Analyser and Architect, while the Coding Router allocates multiple Coder subagents in parallel for concurrent module generation, each possessing focused knowledge rather than handling

multiple domains simultaneously. Each agent maintains its own context window, with Shared Memory storing validated intermediate results that agents retrieve as needed, preventing context accumulation in a single window. Critically, the Verification Agent and Tester Agent operate within an independent Validation Layer rather than embedded within generating agents, eliminating self-validation bias. Routers coordinate workflow progression based on validation outcomes: only verified results advance to subsequent stages, while failures trigger automated correction loops with diagnostic feedback. This architecture addresses domain knowledge specialisation, phase-specific context management, and parallel execution requirements.

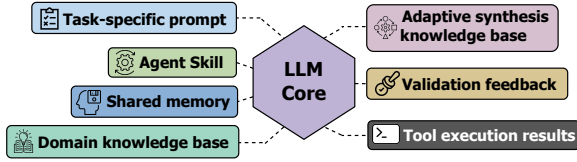


Figure 2: AutoMedImg Agent Context Integration

3.1.2 Agent Context Integration. AutoMedImg achieves comprehensive agent context engineering through seven core components, as illustrated in Figure 2. Task-specific prompts define each agent’s objectives and expected outputs within the workflow. Agent Skills provide structured execution capabilities that guide agents in performing specific tasks step-by-step with precise instructions. Domain knowledge bases supply agent-specific expertise for generation and validation. Shared memory stores validated intermediate results accessible across agents. Adaptive synthesis knowledge base maintains validated pipelines from previous projects. Validation feedback provides diagnostic information guiding iterative corrections. Tool execution results capture outputs from command-line or Model Context Protocol (MCP) tools invoked during execution.

Specific resource assignments vary by agent role. Planning Router and Coding Router employ orchestration Skills for workflow coordination: Planning Router sequentially invokes Dataset Analyser and Architect subagents, while Coding Router concurrently allocates multiple Coder subagents for parallel module generation. Dataset Analyser, Architect, and Coder each access specialised knowledge bases: The Dataset Analyser’s knowledge base is derived from nnU-Net [18] dataset card specifications, capturing standardised medical imaging dataset properties; The Architect’s knowledge base is manually curated from leading neural network architectures on [Grand Challenge](#), providing validated pipeline design patterns across diverse medical imaging tasks, with CSP# modeling Skill for converting architecture designs to formal specifications; The Coder’s knowledge base is extracted from official MONAI API documentation, supplying library-specific implementation templates. The Verification Agent’s knowledge base comprises manually summarised ISO/IEC 25023 modularity standards and IEC 62304 medical device software conventions, providing structured validation criteria and scoring rubrics for semantic assessment, CSP# interpretation knowledge base for understanding verification outputs, alongside Skills for invoking PAT tool for formal verification. Tester accesses

the Testing Knowledge Base containing test case patterns and coverage criteria, while utilising static analysis tools (Pylint, Radon) and dynamic testing frameworks (Pytest) through MCP. All agents share access to shared memory for validated intermediate results and adaptive synthesis knowledge base for cross-project reference. Validation feedback from failed checks and tool execution results is incorporated during the fix-loop, enabling targeted corrections. This comprehensive context integration enables each agent to automatically construct domain-appropriate context without manual prompting.

3.2 Validation-Based Context Engineering

Traditional LLM-based code generation operates through prompt-and-generate cycles, where agents produce outputs solely based on input prompts and pre-trained knowledge. This approach suffers from three critical limitations: (1) without checking outputs, agents cannot assure output quality and reliability; (2) outputs propagate unchecked, allowing errors to cascade through subsequent stages, and (3) without memory management through systematic validation, high-quality patterns may not be reliably adapted or reused across projects. In medical image processing, these limitations manifest as misaligned preprocessing sequences, inconsistent architecture designs, and functionally incorrect implementations.

AutoMedImg addresses these limitations through validation-based context propagation: only outputs that satisfy validation criteria propagate to subsequent agents, while failures generate diagnostic feedback, enabling targeted refinement. The mechanism operates across two phases: the Planning Phase validates dataset analyses and architecture designs through dataset analysis and planning semantic validation; the Coding Phase validates module implementations and system assembly through static and execution validation. We formalise this mechanism through the following definitions:

Definition 1 (Agent Context). The context of agent A_i is defined as:

$$C_{A_i} = \langle P_i, M, K_D^i, K_{Ada}, H_i, V_i \rangle \quad (1)$$

where P_i represents agent A_i ’s task-specific prompt; M denotes shared memory containing accumulated validated outputs from all predecessor agents; K_D^i is agent A_i ’s domain knowledge base; K_{Ada} is the Adaptive Pipeline Synthesis Knowledge Base; H_i is agent A_i ’s tool or Skill execution history; and V_i is the agent’s accumulated validation feedback.

Definition 2 (Context Evolution). Agent A_i generates output $o_i = g_{A_i}(C_{A_i})$ from its context, which is then evaluated by validation function $\mathcal{V} : \mathcal{O} \rightarrow \{\text{pass}, (\text{fail}, f)\}$. On pass, o_i is stored in shared memory M and the next agent A_{i+1} proceeds with updated context; on fail, diagnostic feedback f_i is appended to V_i and A_i retries with enriched context, enabling targeted correction.

3.2.1 Domain Knowledge and Shared Memory. Domain knowledge bases K_D^i and shared memory M serve as foundational context components enabling validation-based context engineering across both Planning and Coding phases. Domain knowledge provides agent-specific expertise without requiring manual specification, while shared memory enables the propagation of validated information without redundant computation.

Domain Knowledge for Expertise Provision. Each agent accesses a domain knowledge base K_D^i tailored to its role, providing specialised contextual knowledge during generation. Each agent's K_D^i is constructed from general medical-domain standards and documentation as a knowledge graph using the Cogne framework [35], enabling natural language querying via MCP tool invocation during generation, and covering the most common medical imaging tasks (supervised/semi-supervised segmentation and classification, 2D/3D), modalities (CT, MRI, X-ray), and architectures. For example, the Dataset Analyser's K_D^i encodes properties such as: *"CT images require HU windowing in range [-1000, 3000]; voxel spacing must be extracted and preserved for physical scale consistency."* The Architect's K_D^i stores patterns such as: *"3D anisotropic data requires physical spacing-aware resampling; multi-organ segmentation with class imbalance requires Dice + Cross-entropy combined loss."* The Verification Agent's K_D^i contains criteria such as: *"Ensure patient-level splitting to prevent data leakage; Validate CT preprocessing includes orientation normalisation before resampling."* Without domain knowledge, the agent's outputs lack domain-specific constraints, leading to inappropriate outputs.

Shared Memory for Context Propagation. Shared memory M is implemented via Eion MCP tool, a persistent key-value store for multi-agent coordination, where validated outputs are written after each successful validation and retrieved through natural language queries, enabling flexible and precise access to verified information without redundant derivation. For instance, after the Dataset Analyser completes validated analysis, M contains: modality (CT), dimensions (512×512×196), voxel spacing (0.8×0.8×3.0mm), intensity range (-1000 to 3000 HU), and task type (supervised segmentation). The Architect retrieves these directly to design a patch-based 3D pipeline with HU windowing preprocessing. Similarly, validated module code propagates through M to assembly, eliminating redundant validation and ensuring consistency across workflow stages.

3.2.2 Planning Phase Context Engineering. The Planning Phase constructs validated architecture designs through sequential agent invocation with validation-guided refinement. Dataset Analyser extracts medical imaging characteristics; Architect designs pipeline architectures grounded in validated dataset analysis; Verification Agent ensures planning correctness through two-level semantic validation: dataset analysis semantic checking verifies completeness and domain compliance; planning semantic validation checks architecture correctness through contextual domain conventions and formal CSP# model checking.

Dataset Analysis Semantic Check. Semantic validation $\mathcal{V}_s$ verifies structural completeness and domain compliance of the dataset analysis:

$$\mathcal{V}_s(o_{DA}) = \text{pass} \Leftrightarrow \forall p \in \mathcal{P}_{DA} : p \in o_{DA} \wedge \text{consistent}(o_{DA}) \quad (2)$$

where $\mathcal{P}_{DA} = \{\text{modality, dimensions, spacing, intensity, count}\}$ denotes the required dataset properties. In practice, the Verification Agent queries its domain validation knowledge base and evaluates the dataset analysis JSON output, checking for the presence and consistency of each required property in $\mathcal{P}_{DA}$. For example, for a CT dataset, validation checks that modality is correctly identified, voxel spacing is extracted (e.g., 0.8×0.8×3.0mm), HU intensity range is

reported (e.g., -1000 to 3000), and case counts are consistent. A missing spacing value or inconsistent case count triggers re-analysis with targeted diagnostic feedback. This check is conducted by the Verification Agent through rule-based semantic assessment: the agent queries its domain validation knowledge, which contains structured validation criteria and scoring rubrics, systematically evaluates each architectural decision against these specifications, and generates detailed diagnostic reports that enable targeted refinement of failed components.

Planning Semantic Validation. Contextual semantic validation checks two criteria. $\text{compatible}(\cdot, M)$ verifies that the architecture aligns with validated dataset characteristics in shared memory – for instance, anisotropic volumetric data requires patch-based strategies and voxel-space resampling to preserve physical scale, while multi-organ segmentation requires loss functions that address class imbalance from organ volume distributions. On the other hand, $\text{valid_design}(\cdot, K_D^{\text{Arch}})$ ensures compliance with ISO/IEC modularity requirements and medical imaging conventions drawn from the domain knowledge base, including HU windowing for CT, z-score normalisation for MRI, and patient-level data splitting to prevent leakage. For example, proposing a 2D UNet for a 3D CT dataset, or applying generic resize instead of voxel-space resampling, triggers a compatibility failure with diagnostic feedback specifying the violated convention, enabling targeted architecture revision. This validation is executed in the same procedure as $\mathcal{V}_s$.

Formal semantic validation converts the architecture to a CSP# specification via dedicated Skill and validates it via PAT tool:

$$\mathcal{V}_{fs}(o_{Arch}) = \text{PAT}(\text{CSP}(o_{Arch})) \quad (3)$$

verifying deadlock-freedom, race condition absence, and LTL properties such as $\Box(\text{model_initialized} \rightarrow \Diamond \text{training_finished})$ and $\Box(\text{test_data_loaded} \rightarrow \Diamond \text{inference_complete})$. Properties are derived from manually verified templates that specify standard safety and liveness requirements for medical imaging workflows. The Verification Agent adapts these templates to the current project through regex-based Skill invocation, substituting project-specific process names and parameters while preserving the verified safety and liveness properties. Since medical image processing pipelines follow consistent algorithmic patterns (data loading, preprocessing, training, inference), the templates comprehensively cover the required properties across different tasks.

Through two-level semantic validation on architecture design, the Planning Phase achieves validation-based context engineering: each agent iteratively refines its outputs through validation feedback, and only verified outputs enter shared memory M for subsequent agents, preventing error propagation while eliminating manual specification requirements.

3.2.3 Coding Phase Context Engineering. The Coding Phase translates the validated architecture design into executable code through validation-guided synthesis. Upon receiving $o_{Arch} \in M$ from the Planning Phase, the Coding Router parses the JSON-structured architecture design, which explicitly specifies class definitions with inheritance relationships, method signatures with type annotations, attributes with pre-defined values (e.g., learning rate, patch size), and inter-module dependencies. The Router then decomposes the

design into module specifications for parallel Coder agents to implement. This phase employs three validation mechanisms operating at increasing scopes: static checking validates the quality of individual module code; execution validation verifies the functional correctness of modules through dynamic testing; and assembly validation ensures the consistency of system-level integration.

Static Checking on Module Code. Static checking $\mathcal{V}_{stat}$ verifies syntactic correctness, dependency consistency, and code quality before execution. The Tester Agent employs Pylint for syntax validation and Radon for complexity measurement:

$$\mathcal{V}_{stat}(code_i) = \text{pass} \Leftrightarrow \bigwedge_{m \in \{\text{Pylint}, CC, MI\}} \text{threshold}(m, code_i, \tau_m) \quad (4)$$

where $\text{Pylint}(\cdot)$ measures syntactic correctness and dependency validity, $CC(\cdot)$ denotes cyclomatic complexity, $MI(\cdot)$ represents maintainability index, and τ_m denotes the quality threshold for each metric m gated by ISO/IEC 25023 standards. In practice, the Tester Agent invokes Pylint and Radon via MCP tool calls, collecting their structured outputs and evaluating CC and MI scores against predefined thresholds ($\tau_{CC} = \tau_{MI} = 80$), while Pylint pass/fail is determined automatically by its built-in rule set. Static checking eliminates syntactic errors before execution, reducing the complexity of runtime validation. Failed checks enrich V_{Coder} with diagnostic feedback including error locations, dependency conflicts, and API mismatches for targeted correction.

Execution Validation on Module Code. Execution validation $\mathcal{V}_{exec}$ verifies runtime correctness through dynamic testing via Pytest. The Tester generates test cases covering three categories: assumption tests validate input constraints (e.g., verifying input tensor shape matches expected dimensions $[B, C, D, H, W]$ for 3D volumetric data); functional tests verify computational correctness (e.g., checking that preprocessing output intensity range falls within the expected HU window after normalisation); and integration tests check interface compatibility (e.g., confirming that preprocessed output shape matches NN module input expectations). Formally:

$$\mathcal{V}_{exec}(code_i) = \bigwedge_{t \in \text{Tests}(code_i)} \text{execute}(t, code_i) = \text{expected}(t) \quad (5)$$

The Tester Agent generates 30–40 test cases per module via LLM invocation, guided by testing rules and examples from its knowledge base. Test cases are executed via MCP tool calls to Pytest, with execution traces captured and appended to V_{Coder} as diagnostic feedback for targeted correction. Failed tests trigger refinement cycles where execution traces reveal runtime behaviour, enabling targeted debugging. Only modules satisfying both static and execution validation propagate to assembly.

Assembly Validation. Assembly validation $\mathcal{V}_a$ verifies system-level correctness after module integration by applying both static checking and execution validation to the assembled pipeline. While individual modules may pass isolated validation with mocked inputs, module-level correctness does not guarantee system-level correctness when real data flows among modules. For example, the Dataset module may correctly output a dictionary with key image that passes its own validation, while the NN module expects key data, violating the interface contract that only manifests when both modules are assembled. Assembly validation detects such interface

mismatches, dimension incompatibilities, and incorrect sequencing, triggering targeted refinement with diagnostic feedback.

Through static, execution, and assembly validation, the Coding Phase achieves multi-scale validation-based context engineering: individual modules refine through validation feedback, validated modules propagate through M_{code} , and only high-performing complete pipelines enter K_{Ada} for future adaptive synthesis, ensuring both correctness and quality throughout implementation.

3.3 Cross-Project Adaptive Pipeline Synthesis

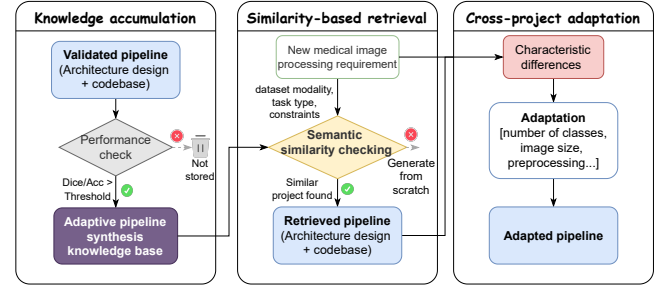


Figure 3: Cross-project Adaptive Pipeline Synthesis Workflow

A critical limitation of LLM-based code generation is the lack of cross-project continuity: each generation session operates independently, without retaining knowledge from previous successful tasks. In medical image processing, analogous scenarios frequently arise: analysing different organs within the same modality, segmenting tissues with similar features, or processing datasets with comparable structures. These recurring patterns necessitate cross-project learning that standard LLMs cannot exploit. Cross-project adaptive pipeline synthesis addresses this by accumulating validated solutions in K_{Ada} and leveraging them to guide future generations. This constrains the solution space from exhaustive assumption to synthesis within validated neighbourhoods, minimising error, and enhancing generation efficiency. Figure 3 illustrates this three-stage process:

Knowledge Accumulation. Successfully validated pipelines achieving satisfactory performance enter K_{Ada} for cross-project reference:

$$K_{Ada} \leftarrow K_{Ada} \cup \{(o_{Arch}, code, \xi, \rho)\} \quad \text{if } \mathcal{V}_a = \text{pass} \wedge \rho \geq \theta \quad (6)$$

where o_{Arch} represents the validated architecture design, $code$ represents the complete pipeline codebase, ξ captures project characteristics (dataset modality, task type, dimensionality, architectural constraints), ρ denotes performance metrics (Dice score, accuracy), and θ defines the quality threshold. In practice, validated pipeline entries are appended incrementally to the agent knowledge graph via Cogne, enabling retrieval through the same natural language querying mechanism as the domain knowledge base; each dataset can only reference pipelines validated from previously processed datasets, ensuring all retrieved components originate exclusively from prior evaluation datasets and avoid information leakage across projects. Each entry preserves the complete pipeline structure, validated components, and achieved performance, enabling future

projects to access proven solutions rather than generating them from scratch.

Similarity-Based Retrieval. When processing a new project with requirements ξ_{new} , AutoMedImg retrieves relevant pipelines from K_{Ada} based on characteristic similarity:

$$\mathcal{R}(\xi_{new}) = \{(o_{Arch}, code, \xi, \rho) \in K_{Ada} \mid sim(\xi_{new}, \xi) \geq \tau_{sim}\} \quad (7)$$

In practice, $sim(\cdot, \cdot)$ is computed via LLM-based semantic matching, where the Router evaluates characteristic alignment between the new project and retrieved entries through natural language reasoning over project descriptors, applying hierarchical matching: hard constraints, including dataset modality and task type, must match exactly, while soft constraints such as dimensionality and architectural requirements are assessed by degree of similarity. τ_{sim} defines the minimum alignment threshold across both hard and soft constraints, where any mismatch in hard constraints immediately disqualifies retrieval regardless of soft constraint similarity. Retrieved pipelines serve as reference implementations, providing validated architecture designs and codebases for adaptation.

Cross-Project Adaptation. For a retrieved entry $(o_{Arch}, code, \xi_{old}, \rho) \in \mathcal{R}(\xi_{new})$, agents extract components and adapt them based on characteristic differences $\Delta(\xi_{old}, \xi_{new})$ such as class numbers, image size, and preprocessing requirements, and adapt the retrieved pipeline accordingly. For example, when processing semi-supervised 3D CT segmentation after a supervised 3D CT segmentation task, AutoMedImg retrieves the segmentation pipeline as a reference, adapts the same data loading strategy, adjusts the number of output classes, and adds the unlabelled data handling logic, while retaining the validated preprocessing pipeline and 3D UNet architecture. All adapted outputs undergo validation in the main pipeline to ensure correctness for the target project.

Through K_{Ada} , AutoMedImg achieves cross-project adaptation: validated pipelines accumulate incrementally, relevant components are retrieved based on project similarity, and adaptations undergo validation ensuring correctness. This mechanism enables knowledge reuse while maintaining quality standards, operating without human supervision.

4 Evaluation

To thoroughly investigate the effectiveness of AutoMedImg, we formulate the following research questions:

RQ1: What are the fundamental limitations of existing code generation approaches when applied to medical image processing tasks?

RQ2: How does AutoMedImg address the fundamental limitations of existing code generation approaches compared to advanced multi-agent and automated ML frameworks?

RQ3: How do AutoMedImg’s individual validation and context engineering components contribute to reducing LLM errors and ensuring code correctness?

RQ4: How does AutoMedImg’s multi-agent architecture improve code generation quality compared to applying the same context engineering techniques in a single-agent system?

This section proceeds as follows: Section 4.1 presents the benchmark datasets and their features. Section 4.2 describes experimental

configurations and evaluation metrics. Section 4.3 reports experimental results organised by research question, including comprehensive ablation studies.

4.1 Benchmarking Datasets

To evaluate the performance of AutoMedImg and baseline methods across different medical image processing sub-tasks, we selected four task categories comprising six datasets in total:

FLARE22 [33]: A *semi-supervised 3D segmentation* dataset for 13 abdominal organs in CT scans (50 labelled, 2000 unlabelled cases, 512×512×196), challenging due to the large unlabelled pool with limited supervision.

ACDC [4]: A *supervised 3D segmentation* dataset for three cardiac structures in MRI (215×228×10), challenging due to significant anisotropic voxel spacing.

BTCV [24]: A *supervised 3D segmentation* dataset for 13 organs in CT scans (50 cases, 512×512×123), challenging due to limited samples and imbalanced organ volumes.

FLARE21 [32]: A *supervised 3D segmentation* dataset for 4 abdominal organs in CT scans (361 cases, 512×512×126), representing a relatively simpler segmentation task than FLARE22.

Promise12MSBench [23]: A *supervised 2D segmentation* dataset for binary prostate segmentation in MRI (512×512), derived from PROMISE12 [29].

OrganMNIST3D [51]: A *supervised 3D classification* dataset for 11 organ categories in CT scans (1,742 samples, 64×64×64), the most straightforward task in our benchmark.

These six datasets are selected because they cover the two most fundamental medical image processing tasks (segmentation and classification), are publicly available and widely adopted benchmarks, and span a comprehensive difficulty gradient from simple supervised classification to complex 3D semi-supervised segmentation, enabling robust evaluation across diverse scenarios.

4.2 Evaluation Settings & Criteria

4.2.1 Baseline Prompt Design and Configuration. To evaluate the fundamental limitations of existing code generation approaches on medical image processing tasks, all baseline methods receive carefully designed prompts that provide equivalent task structure and necessary tool access, enabling them to attempt the same four-stage pipeline as AutoMedImg: **Dataset Analysis:** Generate scripts to examine dataset file structure, image dimensions, modality characteristics and intensity distributions. **Architecture Design:** Design the complete pipeline architecture based on dataset analysis results, including preprocessing and data-loading strategies, neural network structure, training configurations, and inference workflow. **Module Implementation:** Implement individual modules according to the architecture design. **Pipeline Assembly:** Generate an assembly script integrating all modules into an executable end-to-end pipeline.

Baselines also receive command-line execution permissions and file system access where their architecture permits, enabling them to: (1) execute Python scripts for runtime debugging, (2) read/write files during code generation, (3) check and install required dependencies, and (4) attempt to iteratively refine code based on execution feedback. This configuration allows us to identify what types of

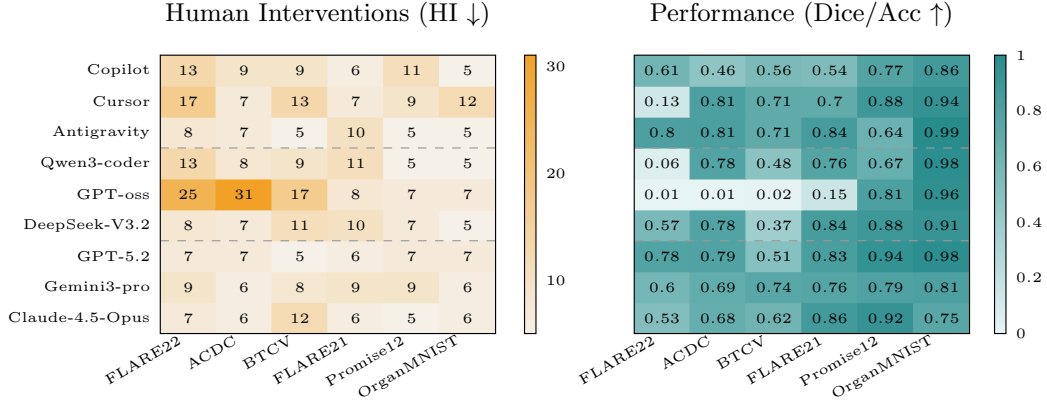


Figure 4: Performance of Code Generation Tools and Base LLMs on Medical Image Processing Tasks

errors and limitations emerge without AutoMedImg’s validation-based context engineering and multi-agent coordination.

4.2.2 Experiment Environmental Settings. The evaluation was performed on an Ubuntu 22.04 server equipped with an AMD Ryzen Threadripper PRO 3995WX CPU (64 cores), 252GB RAM, and 2 NVIDIA RTX A6000 GPUs (48GB VRAM each). We provide a unified Python environment with pre-installed medical imaging libraries and common dependencies. All methods can install additional dependencies as needed during code generation. Regarding AutoMedImg’s domain knowledge, it’s constructed from general medical-domain standards independent of the evaluation datasets selected for this study, ensuring no dataset-specific knowledge is embedded in the framework prior to evaluation.

4.2.3 Evaluation Criteria. We evaluate methods across three dimensions. To measure **autonomy**, we count human interventions (HI) — the number of times a human must intervene to correct errors or provide guidance that the system cannot autonomously resolve, including error correction, design guidance, and workflow instructions. HI takes three forms: direct prompt interactions (providing instructions, relaying error messages, or corrections), passing templates or skeleton code, and, rarely, manual script or configuration updates. For model **performance**, we use the Dice coefficient [53] for segmentation and accuracy for classification, both measured on test sets using identical evaluation protocols. For implementation **quality**, we analyse structural designs including preprocessing strategies, data augmentation techniques, neural network architectures, and computational efficiency (GPU memory usage and training time). On selected datasets, we conduct detailed comparisons to assess whether generated pipelines adhere to medical imaging practices and properly handle volumetric data properties. All experiments are conducted with Pass@3, running each configuration three times and reporting the best-performing trial.

4.3 Experimental Results

4.3.1 Limitations of Existing Code Generation Approaches. To identify the fundamental limitations of existing code generation approaches on medical imaging tasks (RQ1), Table 4 presents the

evaluation of existing baseline coding tools and LLMs on benchmarking medical image processing tasks.

Proprietary IDE tools require 5–17 human interventions across all tasks, revealing a fundamental inability to autonomously handle domain-specific requirements. Interventions predominantly take the form of design guidance and preprocessing specification, such as selecting appropriate architectures, applying correct preprocessing, and accounting for volumetric data properties, as well as error correction for frequent execution failures. While these tools can generate syntactically correct code, they lack domain knowledge to make appropriate design decisions, such as selecting appropriate architectures, applying appropriate preprocessing, and accounting for volumetric data properties, leading to frequent failures that require human correction. Notably, Antigravity performs most competitively (0.6372–0.9938, fewer than 10 interventions) owing to its partial memory retention across sessions, indirectly suggesting the value of cross-project knowledge accumulation. **Open-source LLMs** exhibit severe limitations in both domain knowledge and autonomous error correction. Interventions span the broadest range, including domain design guidance, preprocessing specification, manual script execution, and error message relay. Without domain guidance, these models make critical preprocessing errors that break fundamental medical imaging properties. For example, GPT-oss applies unrestricted Resize instead of voxel-space Resample, corrupting physical scale information and causing near-zero segmentation performance (0.0067 Dice on FLARE22), yet lacks the validation mechanisms to detect and correct such errors autonomously. **Proprietary LLMs** demonstrate stronger baseline capabilities but still require 5–12 interventions with inconsistent performance across datasets (0.5058–0.9752 Dice), revealing that general coding ability alone is insufficient for full autonomy in specialised domains even with state-of-the-art models. Interventions are more mixed, spanning both domain design guidance and workflow instructions, reflecting stronger but still insufficient domain awareness.

Answer to RQ1: Existing code generation tools and LLMs consistently fail on medical imaging tasks due to lack of domain knowledge and validation, with Antigravity’s partial memory retention suggesting cross-project knowledge accumulation as a promising direction.

Table 1: Comparison of AutoMedImg Against Multi-agent and AutoML Frameworks on Medical Image Processing Tasks

Method	FLARE22		ACDC		BTCV		FLARE21		Promise12		OrganMNIST	
	HI↓	Dice↑	HI↓	Dice↑	HI↓	Dice↑	HI↓	Dice↑	HI↓	Dice↑	HI↓	Acc↑
Multi-agent frameworks												
CAMEL with GPT5.2 [25]	6	0.6523	7	0.7234	9	0.6312	6	0.7018	6	0.8456	5	0.8923
MetaGPT X [15]	4	0.7132	3	0.7750	3	0.6845	4	0.7534	3	0.8821	3	0.9125
MaCTG [52]	3	0.5104	4	0.6273	2	0.5205	3	0.8414	2	0.8719	1	0.8820
AutoML frameworks												
AutoML-Agent [49]	5	0.7421	2	0.6907	4	0.6237	2	0.7428	1	0.8870	1	0.9752
M ³ builder [9]	6	0.7568	5	0.7413	1	0.7392	3	0.7464	3	0.8437	4	0.8727
Ours												
AutoMedImg-Qwen3-Coder	0	0.7309	0	0.7856	0	0.6423	0	0.8204	0	0.8696	0	0.9752
AutoMedImg-Deepseek-V3.2	0	0.8333	0	0.7943	0	0.7678	0	0.8518	0	0.9293	0	0.9565
AutoMedImg-GPT5.2	0	0.8090	0	0.7901	0	0.7669	0	0.8667	0	0.9396	0	0.9752
AutoMedImg-Gemini3-pro	0	0.8440	0	0.8936	0	0.7664	0	0.8945	0	0.9347	0	0.8509
AutoMedImg-Claude-4.5-Opus	0	0.8185	0	0.9041	0	0.7696	0	0.8789	0	0.9517	0	0.9938

4.3.2 Performance of AutoMedImg against Advanced Multi-agent and AutoML Frameworks. To demonstrate how AutoMedImg addresses the limitations (RQ2), we compare AutoMedImg against multi-agent and AutoML frameworks under identical task configurations, providing both quantitative performance evaluation (Table 1) and qualitative pipeline design analysis on BTCV (Table 2).

Multi-agent frameworks demonstrate improved autonomy over baseline tools but still suffer from the identified fundamental limitations. CAMEL lacks domain-specific guidance, resulting in basic preprocessing and augmentation strategies. MetaGPT X still shows performance gaps on complex tasks (0.6845 Dice on BTCV, 0.7750 on ACDC), demonstrating that code generation quality alone is insufficient without domain validation. MaCTG achieves the lowest intervention counts (1–4) through intra-agent validation, yet without domain knowledge its validation fails on complex multi-organ segmentation (0.5104 on FLARE22, 0.5205 on BTCV) despite strong performance on simpler tasks, confirming that multi-agent architecture alone is insufficient without domain-specific validation. **AutoML frameworks** exhibit different but equally limiting constraints. AutoML-Agent requires a manually provided code skeleton for each new task type, fundamentally constraining its applicability, and its retrieval mechanism fetches neural network architectures while ignoring preprocessing requirements for complex medical data formats, increasing interventions on specialised tasks (5 on FLARE22, 4 on BTCV). M³Builder relies on manually provided dataset cards and fixed templates for segmentation and classification, lacking dynamic pipeline design capability. Its template-based approach causes hallucinations on task type selection and aborted executions on tasks outside template coverage, particularly semi-supervised segmentation (6 interventions on FLARE22). Its competitive BTCV performance (0.7392 Dice) stems from predefined template coverage and forced nnU-Net backbone rather than adaptive design capability.

AutoMedImg addresses the identified limitations across both framework categories. Domain-specific knowledge bases eliminate the need for manually provided skeletons or dataset cards, enabling fully automated generation from arbitrary datasets. Multi-stage validation prevents error propagation and hallucinations that cause framework abortions. Adaptive pipeline synthesis enables cross-project knowledge reuse without the rigidity of fixed templates.

AutoMedImg achieves zero human interventions across all task types including semi-supervised segmentation, while maintaining competitive or superior performance compared to all frameworks.

Answer to RQ2: While multi-agent and AutoML frameworks partially address RQ1 limitations, they introduce their own constraints including manual input dependencies and fixed templates. AutoMedImg overcomes these through domain-specific knowledge, systematic multi-stage validation, and adaptive synthesis, achieving fully automated generation where existing frameworks still require human preparation.

4.3.3 Ablation Study. To quantify the contribution of each individual validation and context-engineering component (RQ3), we ablate each component individually while keeping all others intact, measuring the impact on autonomy, performance, and generation efficiency across all six datasets with Claude-4.5-Opus as the backbone (Table 5).

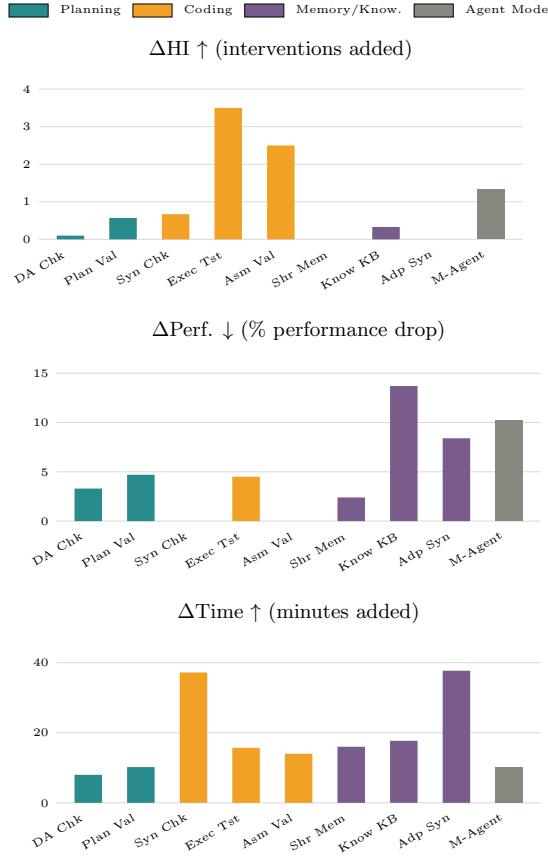
Planning Phase Validations. Dataset analysis semantic checking removal increases interventions by 0.10 with 3.3% performance degradation, as missing critical properties such as voxel spacing or intensity range lead to subtly incompatible downstream designs that only manifest as performance degradation rather than outright failures. Planning semantic validation removal has a more severe impact (+0.57 HI, -4.7%), reflecting the central role of architecture validation in the overall pipeline — without contextual validation, agents may select fundamentally incompatible architectures such as 2D networks for 3D volumetric data. Without formal semantic validation, occasional specification-level inconsistencies that contextual checks cannot detect go unresolved. While medical imaging pipelines follow straightforward sequential structures that rarely violate process ordering properties, formal verification provides an additional correctness guarantee that catches edge cases before implementation begins. Together, these two planning validation steps form a comprehensive correctness gate, preventing a combined +0.67 HI increase and significant performance degradation before implementation begins.

Coding Phase Validations. Code validations exhibit a strong impact on autonomy. Syntax check removal adds 0.67 interventions and increases generation time by 37.2 minutes, as downstream validations eventually catch syntax errors after attempting to execute

Table 2: Detailed Method Comparison on BTCV Dataset (Supervised 3D Segmentation)

Method	Preprocessing	Augmentation	Model	GPU	Time
Multi-agent frameworks					
CAMEL-GPT-5.2	Res., Norm.	Rot., Flip	UNet3D	8.9	146
MetaGPT X	Ori., Res., Norm.	Patch, Rot., Flip, Int.	UNet3D+RN	6.0	257
MaCTG	Ori., Res., Norm.	Rot., Noise, Flip, Int., Smooth	UNet3D	25.5	203
AutoML frameworks					
AutoML-Agent	Ori., Res., Norm.	Patch	UNet3D	14.4	106
M ³ builder	Norm., Pad	Patch, Rot., Noise, Flip, Int.	nnUNet	15.8	235
Ours					
AutoMedImg-Qwen3-coder	Ori., Res., Norm.	Patch, Rot., Flip	UNet3D	9.2	318
AutoMedImg-DeepSeek3.2	Ori., Res., Norm.	Rot., Flip, Noise, Int.	UNet3D	16.6	63
AutoMedImg-GPT-5.2	Ori., Res., Norm.	Patch, Rot., Flip, Int.	UNet3D	12.5	382
AutoMedImg-Gemini3-pro	Ori., Res., Norm., Pad	Patch, Rot., Flip, Int.	SwinUNet3D	23.1	533
AutoMedImg-Claude4.5-Opus	Ori., Res., Norm., Pad	Patch, Rot., Noise, Flip, Int.	UNet3D	14.9	477

Abbreviations: Ori.=Orientation, Res.=Resample, Norm.=Normalize, Rot.=Rotate, Int.=Intensity, RN=ResNet.
GPU refers to VRAM consumption during training in GB; Time refers to training time in minutes.

**Figure 5: Ablation Study: Average Impact of Component Removal on AutoMedImg with Claude-4.5-Opus**

malformed code. This result demonstrates that early-stage checks prevent costly downstream failures. Execution testing removal dramatically increases HI by 3.50 with 4.5% performance degradation, representing the most critical validation for automated generation.

Without runtime verification, functional errors, such as tensor dimension mismatches or incorrect function calls, can propagate to later stages, requiring substantial human debugging. Assembly validation removal causes 2.50 additional interventions, confirming its necessity for detecting integration issues between individually validated modules.

Memory and Knowledge Components. Shared memory removal causes 2.4% performance degradation with a 16-minute time increase, indicating its role in maintaining context consistency across agents. Removing the knowledge base results in the largest performance degradation (-13.7%) with minimal increase in HI (+0.33), indicating that domain-specific knowledge primarily guides appropriate designs. Without medical imaging practices and conventions, agents generate syntactically correct but suboptimal pipelines, such as using torchvision transforms instead of MONAI medical imaging transforms, or applying a generic resize operation that breaks voxel spacing information. Adaptive synthesis removal results in 8.4% performance degradation and an over 37-minute time overhead, confirming that cross-project learning enables agents to adapt pipelines from previous projects rather than regenerate from zero. Without it, agents must generate solutions independently, resulting in slow generation and suboptimal designs.

Answer to RQ3: AutoMedImg’s validation-based context engineering achieves automated generation through three integrated mechanisms: multi-stage validation for systematic correction, domain knowledge and shared memory for quality and efficiency, and adaptive synthesis for cross-project learning, collectively eliminating manual prompting throughout the coding process.

To isolate the architectural contribution of multi-agent design (RQ4), we compare multi-agent versus single-agent execution under identical context-engineering settings, where the single-agent mode receives the same domain knowledge, validation mechanisms, and shared memory as the full system. AutoMedImg employs a multi-agent architecture to address the fundamental limitations of single-agent systems in complex code generation: domain specialisation, context management across phases, and parallel execution.

Table 3: Comparison of AutoMedImg with Automated Medical Imaging Methods

Method	Complete Pipeline	Task Specific Design	Multi-task Support	Cross-project Reference	Multi-stage Validation	Production-ready Implementation	Fully Automated
NAS [50]	✗ (NN only)	✗ (Depend on search space)	✓	✗	✗	✗	✗ (Predefined search space needed)
nnU-Net [18]	✓	✓	✗ (Seg. only)	✗	✗	✓	✗ (Manual setup needed)
MedSAM [31]	✗ (Inference only)	✗ (Fixed pre-trained model)	✗ (Seg. only)	✗	✗	✓	✗ (Interactive inference)
AutoMedImg	✓	✓	✓	✓	✓	✓	✓

The Multi-Agent (M-Agent) category in Figure 5 demonstrates that removing this architecture significantly degrades performance, validating the necessity of this design choice.

Experiments on AutoMedImg with Claude-4.5-Opus demonstrate that single-agent mode reduces performance by 10.2%, requires 1.33 additional interventions, and adds 10 minutes generation time, illustrating three critical limitations: (1) Self-validation bias occurs when a single agent possesses both generation and validation knowledge, occasionally applying validation criteria during generation then merely confirming its own decisions, allowing errors to propagate unchecked. (2) Context compaction forces the agent to eliminate earlier validated planning outputs when exceeding context limits, causing information loss. (3) Sequential processing adds overhead as modules must be generated sequentially. Multi-agent architecture addresses these limitations through role-based agent separation ensuring independent oversight, phase-separated context management preserving workflow history, and concurrent execution exploiting implementation parallelism.

Answer to RQ4: Multi-agent architecture is essential for automated specialised-domain code generation. Distributed coordination enables specialised reasoning that prevents knowledge confusion, preserves context history across multi-stage workflows, and exploits parallel execution to improve workflow efficiency.

5 Discussion

Qualitative Comparison with Code Generation Baselines. IDE tools and vanilla LLMs possess strong code generation capability but lack domain-specific conventions and built-in validation, resulting in suboptimal pipeline designs and substantial human correction overhead. Multi-agent frameworks provide effective inter-agent cooperation but lack systematic validation and domain grounding, limiting their effectiveness on complex domain-specific tasks. AutoML frameworks offer pipeline automation but require manually provided scaffolds for each new task type, constraining their applicability to predefined task coverage. AutoMedImg addresses these limitations by instantiating base models within a multi-agent architecture augmented with domain KBs and systematic step-wise validation, eliminating the need for manually provided scaffolds through a planning-validation workflow that dynamically synthesises task-appropriate pipelines. Furthermore, AutoMedImg distinguishes itself across all categories through cross-project knowledge accumulation via adaptive pipeline synthesis, enabling validated

solutions to be retrieved and adapted for future tasks, a capability absent in all prior work.

Qualitative Comparison with Medical Imaging Automation. Table 3 compares AutoMedImg with automated approaches in medical imaging and related domains. Medical imaging automation methods (NAS, nnU-Net, MedSAM) incorporate domain expertise but address only partial pipeline components, requiring manual setup or interactive inference. AutoMedImg is the only approach that achieves complete pipeline generation, task-specific design, multi-task support, cross-project knowledge reuse, systematic multi-stage validation and full automation.

Threats to Validity. Despite its advantages, AutoMedImg has certain limitations: 1) Generalisation: AutoMedImg is designed specifically for medical image processing, with domain-specific knowledge bases and agent configurations tailored to this domain. The core framework (multi-agent coordination, validation-based context propagation, shared memory, adaptive synthesis) is however domain-independent and can be extended to other domains requiring systematic dataset analysis and pipeline construction, by constructing domain-specific KBs from relevant standards, defining domain-appropriate semantic validation criteria, and adapting formal verification templates. Adapting to broader general code generation tasks beyond data analysis pipelines could also inherit the framework, but would additionally require adapting agent prompts. 2) Generation Overhead: Multi-agent coordination with iterative validation incurs higher API costs and generation time compared to direct generation. In our experiments, a complete pipeline generation costs approximately \$20–40 with an average runtime of 32 minutes, negligible compared to manual development requiring 4–8 hours at \$200–600.

6 Conclusion

In this work, we present AutoMedImg, a multi-agent code generation framework for medical image processing with validation-based context engineering. By employing multi-agent coordination, multi-stage validation, and comprehensive context integration encompassing domain knowledge, shared memory, validation feedback, and adaptive pipeline synthesis, AutoMedImg achieves fully automated code generation across diverse medical imaging tasks. Evaluation demonstrates that AutoMedImg achieves zero human intervention while maintaining competitive performance. Future work will extend AutoMedImg to additional tasks such as image denoising and registration, and explore improving algorithmic performance beyond code generation.

7 Data Availability

The artifacts that support the results discussed in this paper are available at: <https://github.com/SeanCho1996/AutoMedImg>.

References

- [1] Anthropic. 2025. Introducing Claude Opus 4.5. <https://www.anthropic.com/news/claude-opus-4-5>
- [2] Chetan Arora, John Grundy, and Mohamed Abdelrazek. 2024. Advancing requirements engineering through generative ai: Assessing the role of llms. In *Generative AI for Effective Software Development*. Springer, 129–148. doi:10.1007/978-3-031-55642-5_6
- [3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732
- [4] Olivier Bernard, Alain Lalande, Clement Zotti, Frederick Cervensky, Xin Yang, Pheng-Ann Heng, Irem Cetin, Karim Lekadir, Oscar Camara, Miguel Angel Gonzalez Ballester, et al. 2018. Deep learning techniques for automatic MRI cardiac multi-structures segmentation and diagnosis: is the problem solved? *IEEE transactions on medical imaging* 37, 11 (2018), 2514–2525. doi:10.1109/TMI.2018.2837502
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021). doi:10.48550/arXiv.2107.03374
- [6] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023). doi:10.48550/arXiv.2304.05128
- [7] Tristan Coignion, Clément Quinton, and Romain Rouvoy. 2024. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th international conference on evaluation and assessment in software engineering*, 79–89. doi:10.1145/3661167.3661221
- [8] Google Deepmind. 2025. Gemini 3 Pro. <https://deepmind.google/models/gemini/pro/>
- [9] Jinghao Feng, Qiaoyu Zheng, Chaoyi Wu, Ziheng Zhao, Ya Zhang, Yanfeng Wang, and Weidi Xie. 2025. M 3 builder: A multi-agent system for automated machine learning in medical imaging. In *International Workshop on Agentic AI for Medicine*. Springer, 115–124. doi:10.1007/978-3-032-06004-4_12
- [10] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. InCoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022). doi:10.48550/arXiv.2204.05999
- [11] Github. 2025. GitHub Copilot - Your AI pair programmer. <https://github.com/features/copilot>
- [12] Google. 2025. Introducing Google Antigravity, a New Era in AI-Assisted Software Development. <https://antigravity.google/blog/introducing-google-antigravity>
- [13] Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024. Ds-agent: Automated data science by empowering large language models with case-based reasoning. *arXiv preprint arXiv:2402.17453* (2024). doi:10.48550/arXiv.2402.17453
- [14] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30. doi:10.1145/3712003
- [15] Sirui Hong, Xiaowu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. 2023. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* (2023). doi:10.48550/arXiv.2308.00352
- [16] Andrew Hoopes, Neel Dey, Victor Ion Butoi, John V Guttag, and Adrian V Dalca. 2024. VoxelPrompt: A Vision Agent for End-to-End Medical Image Analysis. *arXiv preprint arXiv:2410.08397* (2024). doi:10.48550/arXiv.2410.08397
- [17] Dong Huang, Jie M Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010* (2023). doi:10.48550/arXiv.2312.13010
- [18] Fabian Isensee, Paul F Jaeger, Simon AA Kohl, Jens Petersen, and Klaus H Maier-Hein. 2021. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature methods* 18, 2 (2021), 203–211. doi:10.1038/s41592-020-01008-z
- [19] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology* 35, 2 (2026), 1–72. doi:10.1145/3747588
- [20] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>
- [21] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479* (2024). doi:10.48550/arXiv.2408.02479
- [22] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. 2023. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, 4015–4026. doi:10.48550/arXiv.2304.02643
- [23] Zeki Kuş and Musa Aydin. 2024. MedSegBench: A comprehensive benchmark for medical image segmentation in diverse data modalities. *Scientific Data* 11, 1 (2024), 1283. doi:10.1038/s41597-024-04159-2
- [24] Bennett Landman, Zhoubing Xu, Juan Igelsias, Martin Styner, Thomas Langerak, and Arno Klein. 2015. Miccai multi-atlas labeling beyond the cranial vault—workshop and challenge. In *Proc. MICCAI multi-atlas labeling beyond cranial vault—workshop challenge*, Vol. 5. Munich, Germany, 12. doi:10.7303/syn3193805
- [25] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 51991–52008. https://proceedings.neurips.cc/paper_files/paper/2023/file/a3621ee907def47c1b952ade25c67698-Paper-Conference.pdf
- [26] Haochen Li, Xin Zhou, and Zhiqi Shen. 2024. Rewriting the code: A simple method for large language model augmented code search. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 1371–1389. doi:10.18653/v1/2024.acl-long.75
- [27] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2024. Acecoder: An effective prompting technique specialized in code generation. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–26. doi:10.1145/3675395
- [28] Xiangrui Li, Paul S Morgan, John Ashburner, Jolinda Smith, and Christopher Rorden. 2016. The first step for neuroimaging data analysis: DICOM to NIfTI conversion. *Journal of neuroscience methods* 264 (2016), 47–56. doi:10.1016/j.jneumeth.2016.03.001
- [29] Geert Litjens, Robert Toth, Wendy Van De Ven, Caroline Hoeks, Sjoerd Kerkstra, Bram Van Ginneken, Graham Vincent, Gwenael Guillard, Neil Birbeck, Jindang Zhang, et al. 2014. Evaluation of prostate segmentation algorithms for MRI: the PROMISE12 challenge. *Medical image analysis* 18, 2 (2014), 359–373. doi:10.1016/j.media.2013.12.002
- [30] Yuqiao Liu, Yanan Sun, Bing Xue, Mengjie Zhang, Gary G Yen, and Kay Chen Tan. 2021. A survey on evolutionary neural architecture search. *IEEE transactions on neural networks and learning systems* 34, 2 (2021), 550–570. doi:10.1109/TNNLS.2021.3100554
- [31] Jun Ma, Yuting He, Feifei Li, Lin Han, Chenyu You, and Bo Wang. 2024. Segment Anything in Medical Images. *Nature Communications* 15 (2024), 654. doi:10.1038/s41467-024-44824-z
- [32] Jun Ma, Yao Zhang, Song Gu, Xingle An, Zhihe Wang, Cheng Ge, Congcong Wang, Fan Zhang, Yu Wang, Yinan Xu, et al. 2022. Fast and low-GPU-memory abdomen CT organ segmentation: the flare challenge. *Medical Image Analysis* 82 (2022), 102616. doi:10.1016/j.media.2022.102616
- [33] Jun Ma, Yao Zhang, Song Gu, Cheng Ge, Shihao Mae, Adamo Young, Cheng Zhu, Xin Yang, Kangkang Meng, Ziyang Huang, et al. 2024. Unleashing the strengths of unlabelled data in deep learning-assisted pan-cancer abdominal organ quantification: the FLARE22 challenge. *The Lancet Digital Health* 6, 11 (2024), e815–e826. doi:10.1016/S2589-7500(24)00154-7
- [34] Wei Ma, Zhihao Lin, Shangqing Liu, Qiang Hu, Ye Liu, Wenhan Wang, Cen Zhang, Liming Nie, Li Li, Yang Liu, et al. 2026. Exploring Code Analysis: Zero-Shot Insights on Syntax and Semantics with LLMs. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3818607
- [35] Vasilije Markovic, Lazar Obradovic, Laszlo Hajdu, and Jovan Pavlovic. 2025. Optimizing the interface between knowledge graphs and llms for complex reasoning. *arXiv preprint arXiv:2505.24478* (2025). doi:10.48550/arXiv.2505.24478
- [36] Peter Mildenberger, Marco Eichelberg, and Eric Martin. 2002. Introduction to the DICOM standard. *European radiology* 12, 4 (2002), 920–927. doi:10.1007/s003300101100
- [37] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. Next: Teaching large language models to reason about code execution. *arXiv preprint arXiv:2404.14662* (2024). doi:10.48550/arXiv.2404.14662
- [38] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022). doi:10.48550/arXiv.2203.13474
- [39] OpenAI. 2025. Introducing GPT-5.2. <https://openai.com/index/introducing-gpt-5-2/>
- [40] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2024. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15174–15186.

- doi:10.18653/v1/2024.acl-long.810
- [41] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. 2024. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714* (2024). doi:10.48550/arXiv.2408.00714
 - [42] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Xiaojiang Chen, and Xin Wang. 2021. A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)* 54, 4 (2021), 1–34. doi:10.1145/3447582
 - [43] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. 2025. Prompt Engineering or Fine-Tuning: An Empirical Assessment of LLMs for Code. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 490–502. doi:10.1109/MSR66628.2025.00082
 - [44] LLM Stats. 2026. MBPP Leaderboard. <https://llm-stats.com/benchmarks/mbpp>
 - [45] SWE-bench. 2026. SWE-bench Leaderboard. <https://www.swebench.com/>
 - [46] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. 2024. A survey on self-evolution of large language models. *arXiv preprint arXiv:2404.14387* (2024). doi:10.48550/arXiv.2404.14387
 - [47] Cursor Team. 2024. Cursor Docs. <https://cursor.com/docs>
 - [48] Price Per Token. 2026. HumanEval Leaderboard. <https://pricepertoken.com/leaderboards/benchmark/humaneval>
 - [49] Patara Trirat, Wonyong Jeong, and Sung Ju Hwang. 2025. AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR, 60099–60146. <https://proceedings.mlr.press/v267/trirat25a.html>
 - [50] Yan Wang, Liangli Zhen, Jianwei Zhang, Miqing Li, Lei Zhang, Zizhou Wang, Yangqin Feng, Yu Xue, Xiao Wang, Zheng Chen, et al. 2024. Mednas: Multiscale training-free neural architecture search for medical image analysis. *IEEE Transactions on Evolutionary Computation* 28, 3 (2024), 668–681. doi:10.1109/TEVC.2024.3352641
 - [51] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. 2023. Medmnist v2-a large-scale lightweight benchmark for 2d and 3d biomedical image classification. *Scientific Data* 10, 1 (2023), 41. doi:10.1038/s41597-022-01721-8
 - [52] Zixiao Zhao, Jing Sun, Zhe Hou, Zhiyuan Wei, Cheng-Hao Cai, Miao Qiao, and Jin Song Dong. 2024. MaCTG: Multi-Agent Collaborative Thought Graph for Automatic Programming. *arXiv preprint arXiv:2410.19245* (2024). doi:10.48550/arXiv.2410.19245
 - [53] Alex P Zijdenbos, Benoit M Dawant, Richard A Margolin, and Andrew C Palmer. 1994. Morphometric analysis of white matter lesions in MR images: method and validation. *IEEE transactions on medical imaging* 13, 4 (1994), 716–724. doi:10.1109/42.363096

Received 2026-03-26; accepted 2026-06-18